

Machine Learning

第八章 基于实例的学习

主要内容

- 基于实例学习的特点
- 主要算法：
 - k-近邻 (k-Nearest neighbor learning) 及变种
 - 局部加权回归 (Locally weighted regression)
 - 径向基函数 (Radial basis function network)
- 概念：
 - 消极学习 (Lazy learning)
 - 积极学习 (eager learning)

8.1 简介

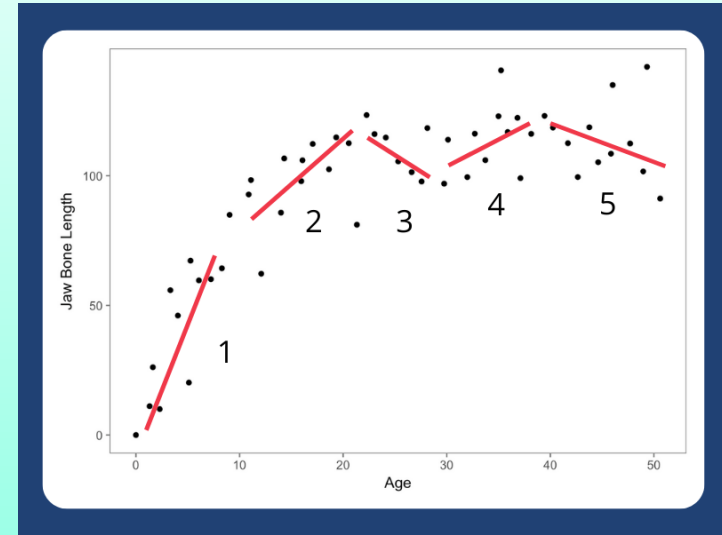
- 基于实例学习特点

- 训练：只存储样本
- 分类：检索训练集中相似的样本



简介(2)

- 与其它算法的关键差异：
 - 为每个分类实例建立目标函数
 - 优点
 - 用多个简单局部函数逼近复杂的目标函数 (局部)
 - 缺点
 - 分类代价高
 - 高效索引训练样本
 - 相似实例可能会距离远
 - 有时目标概念仅依赖于多属性中的几个属性



8.2 k-近邻算法

- 适用问题

- 实例表示为 $\mathbb{R}^n$ 空间中的点，特征向量 $\langle a_1(x), \dots, a_n(x) \rangle$

- 任意两个实例之间的距离可定义：

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^n (a_r(x_i) - a_r(x_j))^2}$$

- 学习目标函数 $f: \mathbb{R}^n \rightarrow V$

- V ：离散值或实数值

k-近邻算法 (2)

- 学习离散值目标函数 $f: \mathbb{R}^n \rightarrow V$, $V = \{v_1, \dots, v_s\}$
 - 训练算法

For each training sample $\langle x, f(x) \rangle$

{ add it to the list *training_examples* }

- 分类算法

- 待分类（查询）实例 x_q
 - 在 *training_examples* 中，选择 K 个 x_q 的最近邻 $x_1 \dots x_k$
 - 返回

$$\leftarrow \underset{\in}{\underset{=}{\text{?}}} \sum \delta \quad \text{?}$$

$$\delta(a, b) = \begin{cases} 1 & a = b \\ 0 & a \neq b \end{cases}$$

k-近邻算法（3）

- 解释：
 - $\hat{y}$: k 近邻中最普遍类别
 - $\hat{y}$: 结果依赖于近邻数目, 见图.8-1（下个PPT）
- k-近邻算法学习到的一般函数（隐含）
 - Fig. 8-1, 1-近邻决策面.
 - Voronoi 图

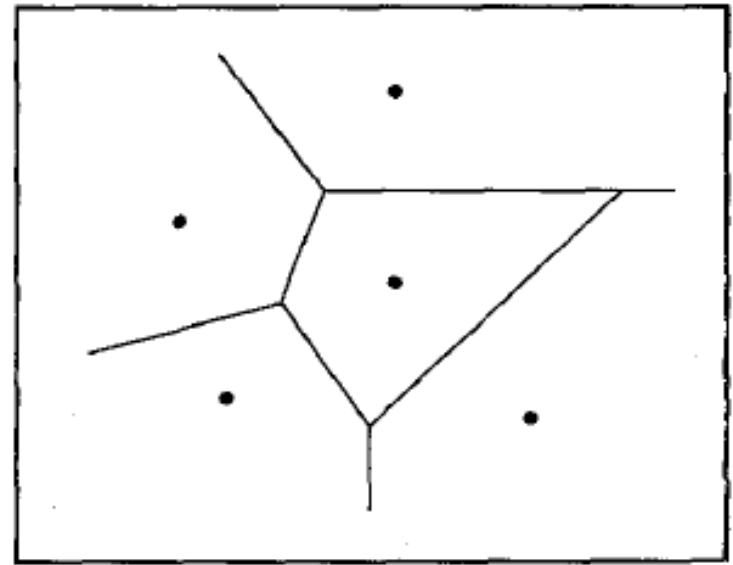
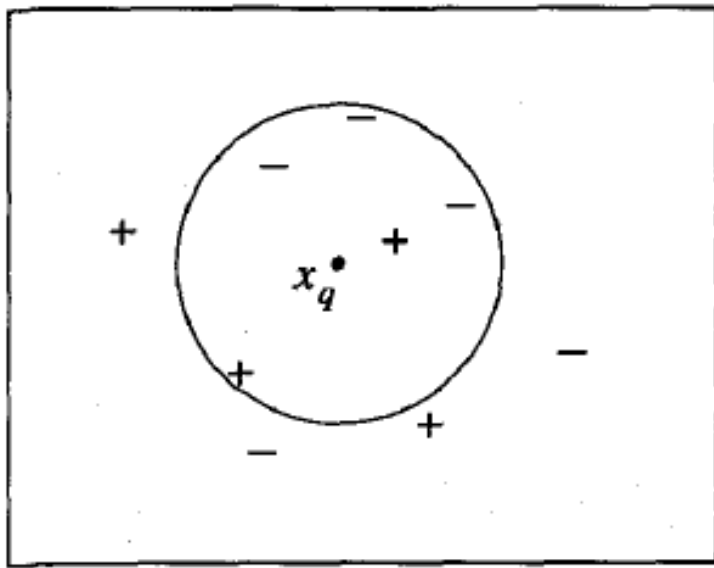
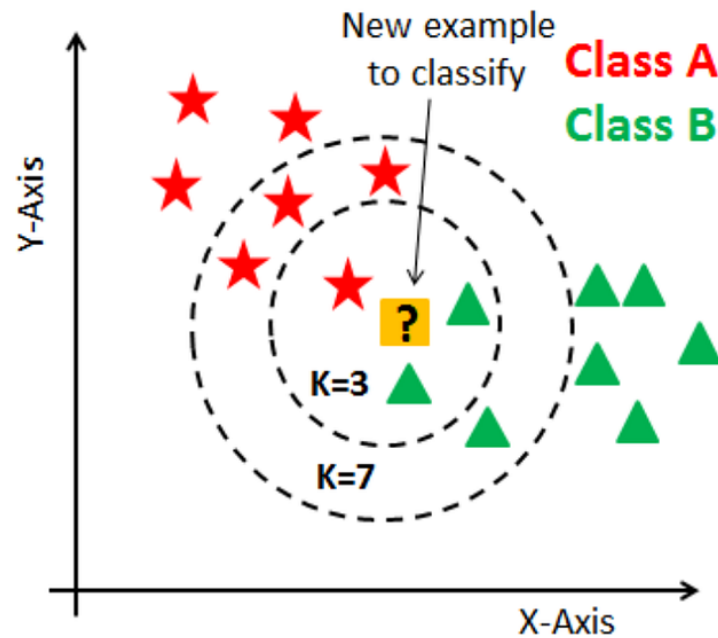


FIGURE 8.1



k-近邻算法 (3)

- 学习实值目标函数
 - 用k个最近邻训练样本的均值表示
 - 实函数 $f: \mathbf{R}^n \rightarrow \mathbf{R}$ 的逼近

$$f(x_q) \leftarrow \frac{\sum_{i=1}^k f(x_i)}{k}$$

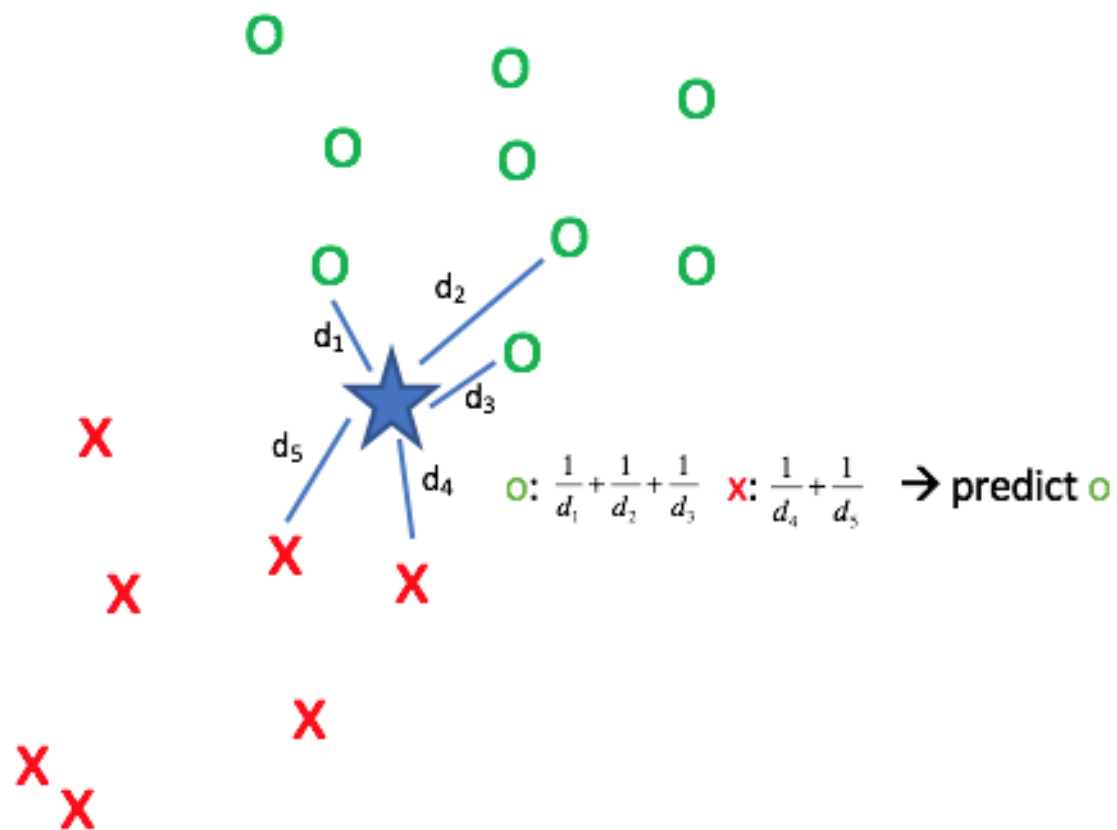
8.2.1 距离加权最近邻算法

- k-近邻改进
 - 根据距离对k-近邻样本进行加权
- 离散值目标函数:

$$\hat{f}(x_q) \leftarrow \arg \max_{v \in V} \sum_{i=1}^k w_i \delta(v, f(x_i)) \quad w_i = \frac{1}{d(x_q, x_i)^2}$$

- 如果 $d(x_q, x_i)^2 = 0$, $\hat{f}(x_q) = f(x_i)$.
 - 如果存在多个零距离样本, $\hat{f}(x_q)$ 为样本数最多的类别
- 实值目标函数:

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^k w_i f(x_i)}{\sum_{i=1}^k w_i} \quad (8.4)$$



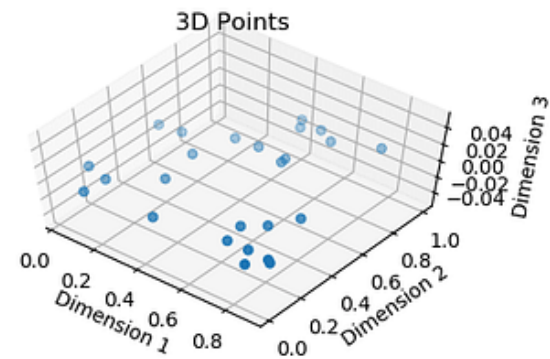
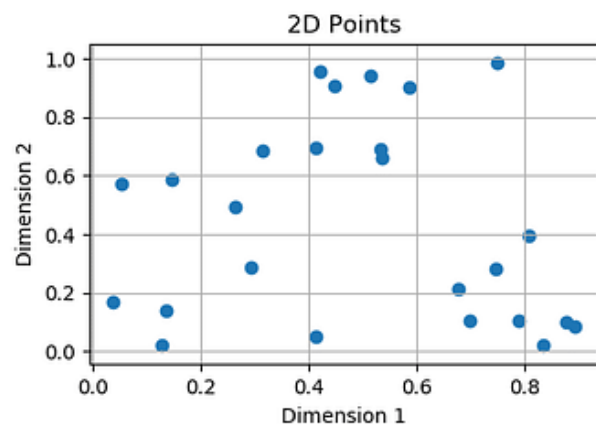
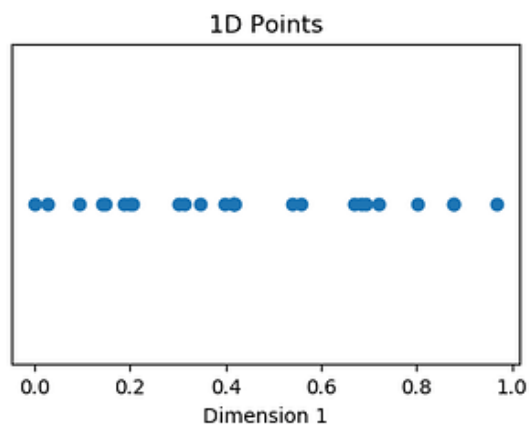
距离加权最近邻算法图示

距离加权最近邻算法（2）

- 改进：用全部训练样本分类查询点
 - 计算全部样本的距离权重
 - 远距离样本对 $\hat{f}(x_q)$ 影响极小
 - 缺点：
 - 分类时间长
 - 全局方法与局部方法
- Shepard 方法
 - 用全局方法逼近实值目标函数（见公式8.4）

8.2.2 k-近邻算法的说明

- 距离加权k-NN 对噪声有鲁棒性
 - 可以平滑掉孤立的噪声训练样本的影响.
- k-近邻的归纳偏置:
 - x_q 的类别与它附近的实例的类别相似.
- k-近邻实践中的问题: 维度灾难 (curse of dimensionality)
 - 基于全部属性计算距离
 - 实际中类别相关属性只占少数.
 - 误导k-近邻分类器

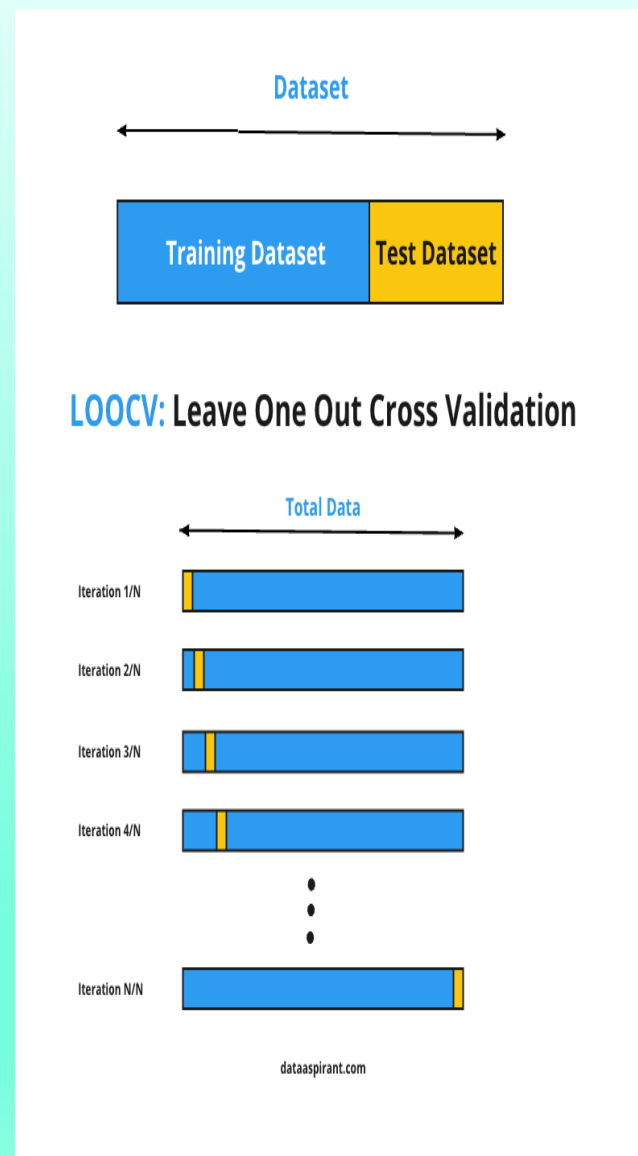


随维数增加，空间中的数据点变得稀疏，距离变大

k-近邻实践中的问题: 维度灾难 (curse of dimensionality)

k-近邻算法的说明 (2)

- 克服维度灾难的方法:
 - 对属性加权, 相当于按比例缩放坐标轴.
 - 伸展第 j 个坐标轴 z_j 倍
 - 用交叉验证, 选取最佳 z_j
 - 清除最不相关属性 ($Z_j=0$)
 - 采用基于留一法 (leave-one-out) 的交叉验证 选择属性子集
 - 用变化的值伸展属性坐标轴
 - 自由度高, 但也增加了过拟合风险



Remarks on K-NN algorithm (3)

- 实践问题: 如何建立搜索近邻样本的高效索引
 - 方法举例
 - KD-tree : 实例存储在叶子节点, 其近邻存储在相同或附近的节点内.
 - 通过测试 x_q 的选定属性, KD tree 把查询点 x_q 排序到相关叶子节点

术语解释

- 统计模式识别术语

- 回归 (Regression) : 逼近实值目标函数
- 残差(Residual): 逼近目标函数的误差 $\hat{f}(x) - f(x)$
- 核函数(Kernel function):
 - 决定训练样例权值的距离函数.
 - $K: w_i = K(d(x_i, x_q))$

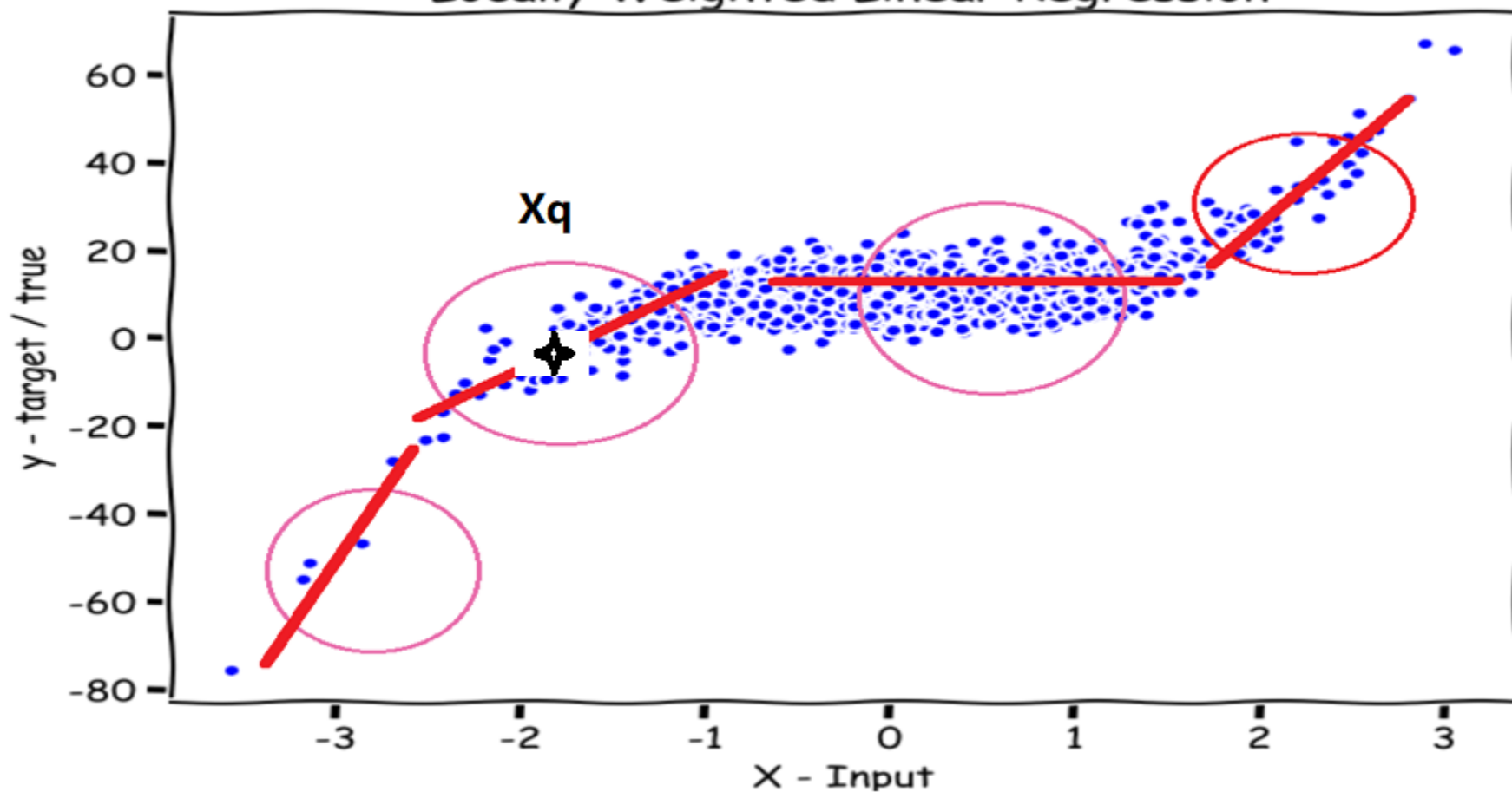
8.3 局部加权回归

- 最近邻方法是在单一查询点 x_q 逼近目标函数
- 局部加权回归是对k-近邻的推广，逼近实值函数
 - 可采用线性函数, 二次函数, 神经网络逼近 f
 - 局部：在 x_q 的局部区域逼近目标函数 f (局部函数)
 - 加权：用加权训练样本构造目标函数 f 的局部近似
 - 回归

局部加权回归 (2)

- 局部加权回归的一般方法:
 - 构造 f 拟合 x_q 邻域内的训练样本 (构造局部逼近函数)
 - 用 f 计算 $f(x_q)$ 的值
 - 删除
$$f$$

Locally Weighted Linear Regression



局部加权线性回归：函数学习的局部特性

8.3.1 局部加权线性回归

- 用线性函数逼近 x_q 邻域的目标函数 f
 - $a_i(x)$: 实例 x 的第 i 个属性

$$f(x) = w_0 + w_1 a_1(x) + \dots + w_n a_n(x)$$

- 定义误差函数

$$E = \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2$$

- 训练规则：本质为Delta规则

$$\Delta w_j = \eta \sum_{x \in D} (f(x) - \hat{f}(x)) a_j(x)$$

局部加权线性回归 (2)

- 三种误差函数 $E(x_q)$

- k-近邻上的误差平方和最小

$$E_1(x_q) = \frac{1}{2} \sum_{x \in k \text{ nearest neighbors of } x_q} (f(x) - \hat{f}(x))^2$$

- D上误差平方和最小, 样例的误差加权 (距离的递减函数)

$$E_2(x_q) = \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2 K(d(x_q, x))$$

- 两种结合

$$E_2(x_q) = \frac{1}{2} \sum_{x \in k \text{ nearest neighbors of } x_q} (f(x) - \hat{f}(x))^2 K(d(x_q, x))$$

局部加权线性回归（3）

- Criterion 2: 也许最好, 计算量大
- Criterion 3: 准则2的近似, 计算代价低
- Criterion 3的梯度公式:

$$\Delta w_j = \eta \sum_{x \in k \text{ nearest neighbors of } x_q} K(d(x_q, x))(f(x) - f(x))a_j(x)$$

8.3.2 局部加权回归的说明

- 可用常量、线性函数、二次函数局部近似目标函数，更复杂的函数形式不常见，因为：
 - 代价高
 - 简单函数的近似效果已相当好

8.4 径向基函数 (Radial Basis Functions)

- 径向基函数
 - 用于函数逼近
 - 类似于距离加权回归、ANN
- 假设表示:

$$f(x) = w_0 + \sum_{u=1}^k w_u K_u(d(x_u, x))$$

- $f(x)$ 是 $f(x)$ 的全局近似函数
 - $K_u(d(x_u, x))$ 是高斯函数

$$K_u(d(x_u, x)) = e^{-\frac{1}{2\sigma_u^2} d^2(x_u, x)}$$

- $K_u(d(x_u, x))$ 有局部化特性

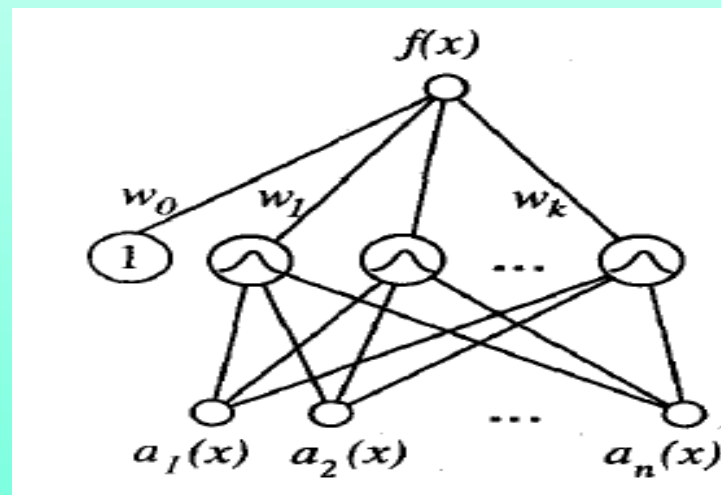
径向基函数 (2)

- $f(x)$ (Gaussian Kernel) 以任意小误差逼近任意函数的条件
 - k 足够大
 - 并且可分别指定每个核的宽度 σ_u^2
- $f(x)$ 可表示为两层神经网络
 - 第一层计算 $K_u(d(x_u, x))$,
 - 第二层是第一层单元的线性组合

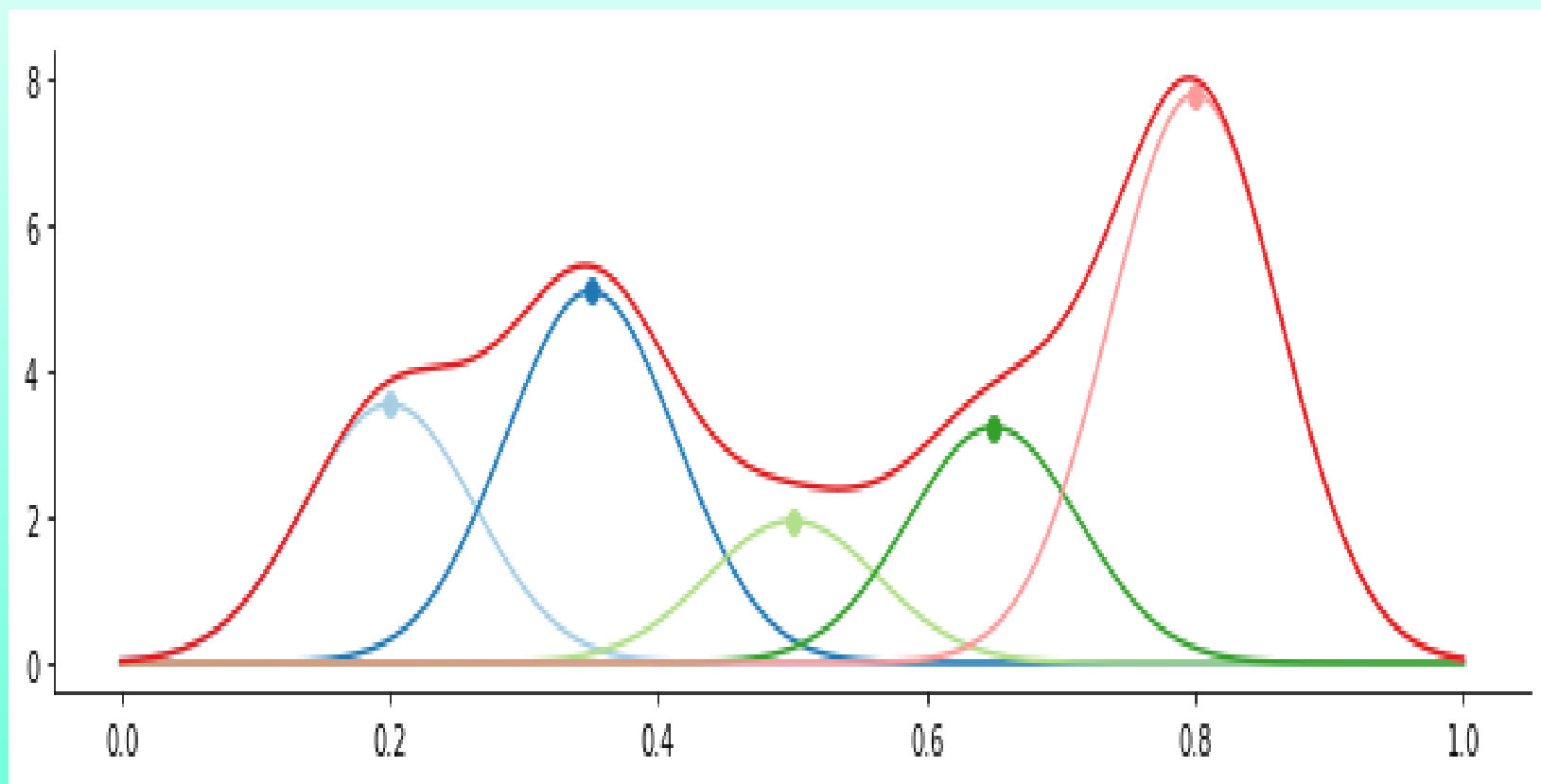
$$f(x) = w_0 + \sum_{u=1}^k w_u K_u(d(x_u, x))$$

- 两阶段训练RBF
 - 确定 k , 选择 x_u 及 σ_u^2
 - 定义全局误差, 学习 权 w_u

$$E = \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2$$



径向基函数的局部特性



径向基函数网络图示

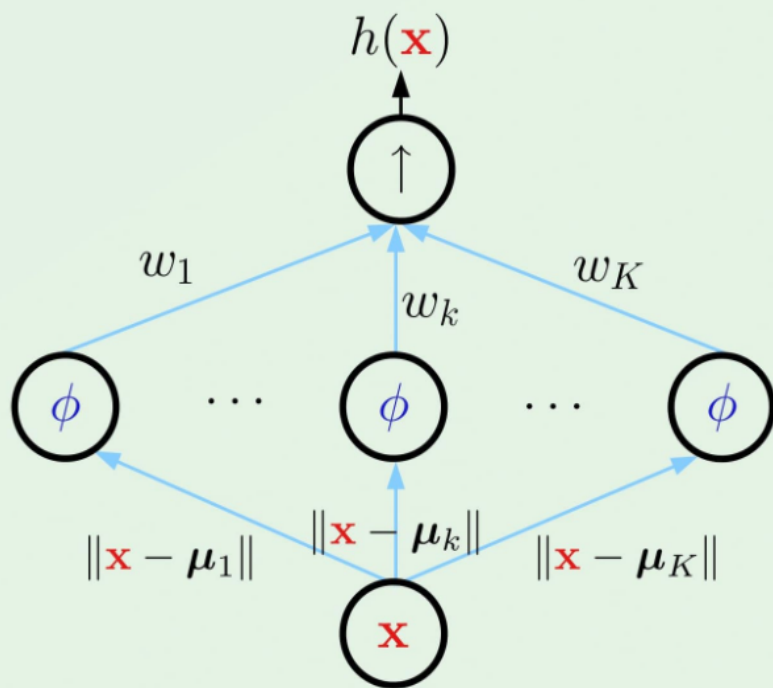
径向基函数 (3)

- 如何选择核函数数量
 - 每一训练样本 $\langle \mathbf{x}_i, f(\mathbf{x}_i) \rangle$ 分配一个
 - $\mathbf{x}_i$ 为中心, σ 相同
 - $\langle \mathbf{x}_i, f(\mathbf{x}_i) \rangle$ 只对 $f(\mathbf{x}_i)$ 产生局部影响
 - 优点 : RBF 可精确拟合训练数据
 - 对每一样本可学习到权 $w_1 \cdots w_m$ 使得

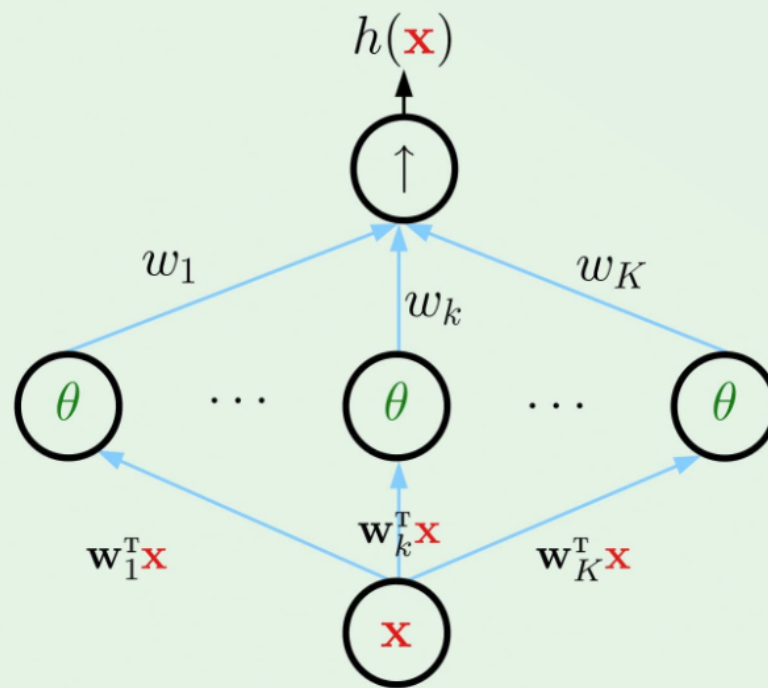
$$f(\mathbf{x}_i) = f(\mathbf{x}_i)$$

径向基函数训练高效

Compare to neural networks



RBF network



neural network

径向基函数（4）

— 核数目优化，使其小于训练样本数

- 目的：提高计算效率
- 核函数的中心在实例空间均匀分布
- 或者, 非均匀分布（特别当实例非均匀分布时）
 - 随机选取训练样本作为核函数的中心
 - 聚类，聚类中心作为核函数的中心

径向基函数(5)

– RBF训练

- 误差函数：误差平方和+正则化项

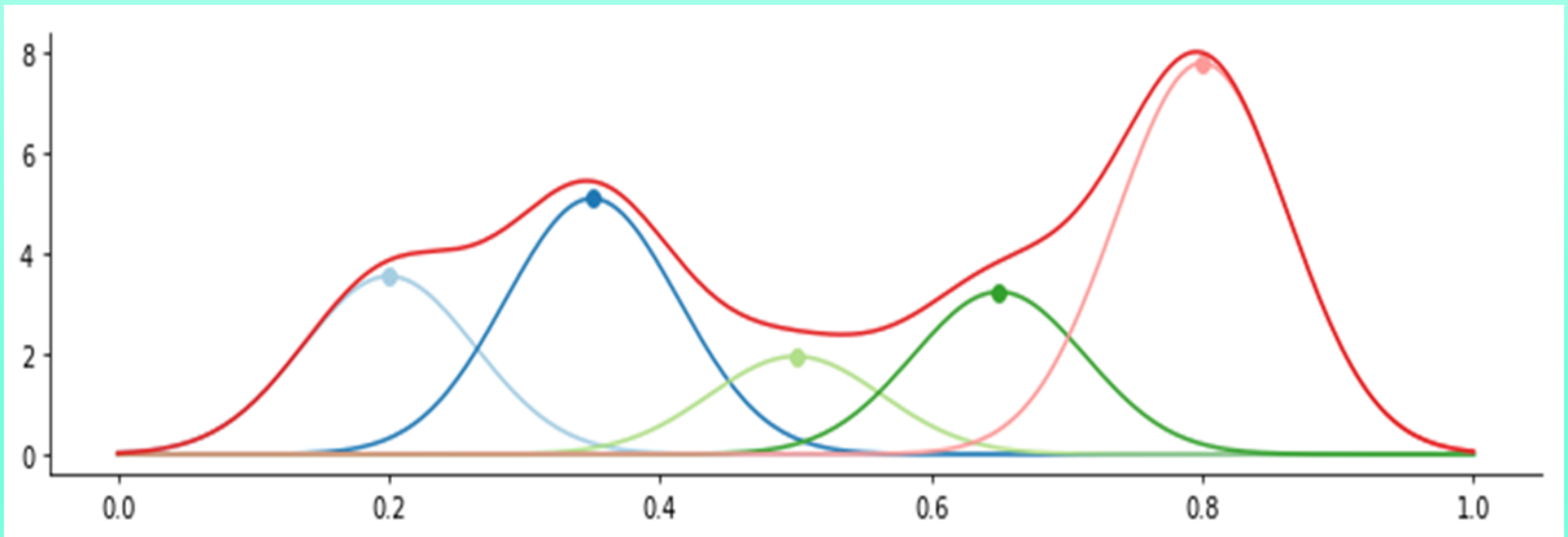
$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - \Phi\mathbf{w}\|^2 + \lambda\|\mathbf{w}_{1:K}\|^2$$

$$w_k \leftarrow w_k + \eta e_i \phi_k(\mathbf{x}_i) - \eta \lambda w_k \quad (\text{for } k = 1, \dots, K)$$

$$e_i = y_i - f(\mathbf{x}_i)$$

径向基函数小结（5）

- 对目标函数的全局逼近
- 输入样本的局部有效性
- 对目标函数进行局部逼近的多个函数的平滑线性组合
- 优点: 训练高效（由于输入层、输出层单独训练）



- RBF可用于分类

RBF for classification

$$h(\mathbf{x}) = \text{sign} \left(\sum_{n=1}^N w_n \exp \left(-\gamma \|\mathbf{x} - \mathbf{x}_n\|^2 \right) \right)$$

Learning: $\sim$ linear regression for classification

$$s = \sum_{n=1}^N w_n \exp \left(-\gamma \|\mathbf{x} - \mathbf{x}_n\|^2 \right)$$

Minimize $(s - y)^2$ on $\mathcal{D}$ $y = \pm 1$

$$h(\mathbf{x}) = \text{sign}(s)$$

8.6 消极学习和积极学习评论

- 消极学习(lazy learning): k-NN, locally weighted regression
- 积极学习(eager learning): RBF
- 两者区别:
 - 计算时间: 训练时间, 分类时间
 - 归纳偏置
 - 泛化时是否考虑查询点 X_q 的信息
- 泛化精度
 - 核心观点:
 - 消极学习器用（查询点附近）多个局部逼近的组合表示目标函数
 - 积极学习器需要在训练时间完成全局逼近函数的学习

消极学习和积极学习评论（2）

- 积极方法能否使用多个局部逼近？
 - 径向基函数网络
- RBF 提供全局逼近
 - 由多个局部核函数的线性组合实现
 - 但是，并非针对查询点的局部逼近

K近邻法的实现：kd树_(补充,选学)

➤ 构造kd树

- Kd树是一种对k维空间中的实例点进行存储以便对其进行快速检索的树形数据结构。Kd树是二叉树，表示对k维空间的一个划分。构造kd树相当于不断地用垂直于坐标轴的超平面将k维空间切分，构成一系列的k维超矩形区域。Kd树的每个结点对应于一个k维超矩形区域。
- 方法：构造根结点，使根结点对应于k维空间中包含所有实例点的超矩形区域；通过递归方法，不断地对k维空间进行切分，生成子结点。在超矩形区域（结点）上选择一个坐标轴和在此坐标轴上的一个切分点，确定一个超平面，这个超平面通过选定的切分点并垂直于选定的坐标轴，将当前超矩形区域切分为左右两个子区域（子结点）；这时，实例被分到两个子区域。这个过程直到子区域内没有实例时终止。在此过程中，将实例保存在相应的结点上。

K近邻法的实现：kd树

➤构造平衡kd树

输入：k维空间数据集 $T = \{x_1, x_2, \dots, x_N\}$,

其中 $x_i = (x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(k)})^T, i=1, 2, \dots, N$;

输出：kd树

1. 开始：构造根结点，根结点对应于包含 T 的k维空间的超矩形区域。选择 $x^{(1)}$ 为坐标轴，以 T 中所有实例的 $x^{(1)}$ 坐标的中位数为切分点，将根结点的超矩形区域切分为两个子区域。切分由通过切分点并与坐标轴 $x^{(1)}$ 垂直的超平面实现。
2. 重复：对深度为 j 的结点，选择 $x^{(l)}$ 为切分的坐标轴， $l = j(\bmod k) + 1$ ，以该结点的区域中所有实例的 $x^{(l)}$ 坐标的中位数为切分点，将该结点对应的超矩形区域切分为两个子区域。切分由

K近邻法的实现：kd树

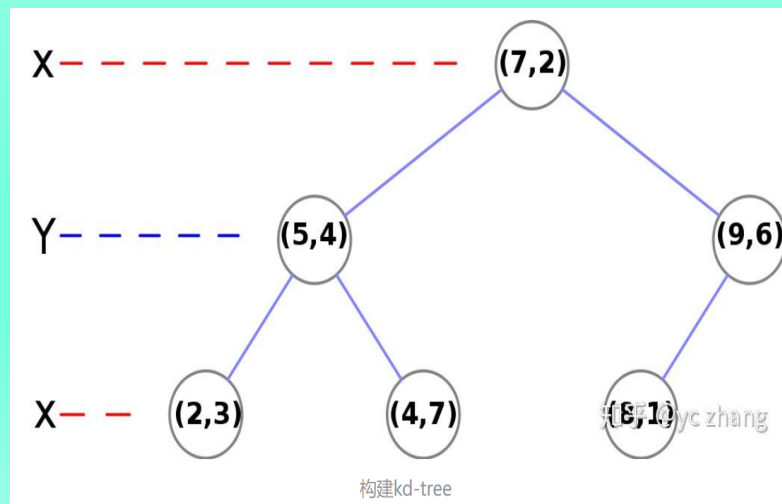
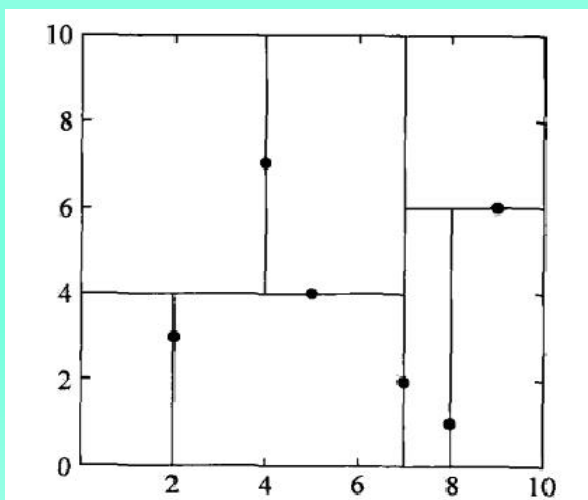
3. 直到两个子区域没有实例存在时停止。从而形成kd树的区域划分。

例：给定一个二维空间的数据集：

$$T = \{(2,3)^T, (5,4)^T, (9,6)^T, (4,7)^T, (8,1)^T, (7,2)^T\}$$

构造一个平衡kd树。

解：根结点对应包含数据集T的矩形，选择 $x^{(1)}$ 轴，6个数据点的 $x^{(1)}$ 坐标的中位数是7，以平面 $x^{(1)}=7$ 将空间分为左、右两个子矩形（子结点）；接着，左矩形以 $x^{(2)}=4$ 分为两个子矩形，右矩形以 $x^{(2)}=6$ 分为两个子矩形，如此递归，最后得到下面所示的**特征空间划分和kd树**。



K近邻法的实现：kd树(补充，选学)

➤ 搜索kd树

- 利用kd树可以省去大部分数据点的搜索，从而减少搜索的计算量。这里以最近邻为例，同样的方法可以应用到K近邻。
- 给定一个目标点，搜索其最近邻。首先找到包含目标点的叶结点；然后从该叶结点出发，依次回退到父结点；不断查找与目标点最邻近的结点，当确定不可能存在更近的结点时终止。这样搜索就被限制在空间的局部区域上，效率大为提高。
- 包含目标点的叶结点对应包含目标点的最小超矩形区域。以此叶点节的实例点作为当前最近点。目标点的最近邻一定在以目标点为中心并通过当前最近点的超球体内部。然后返回当前结点的父结点，如果父结点的另一子结点的超矩形区域与超球体相交，那么在相交的区域内寻找与目标点更近的实例点。如果存在这样的点，将此点作为新的当前最近点。

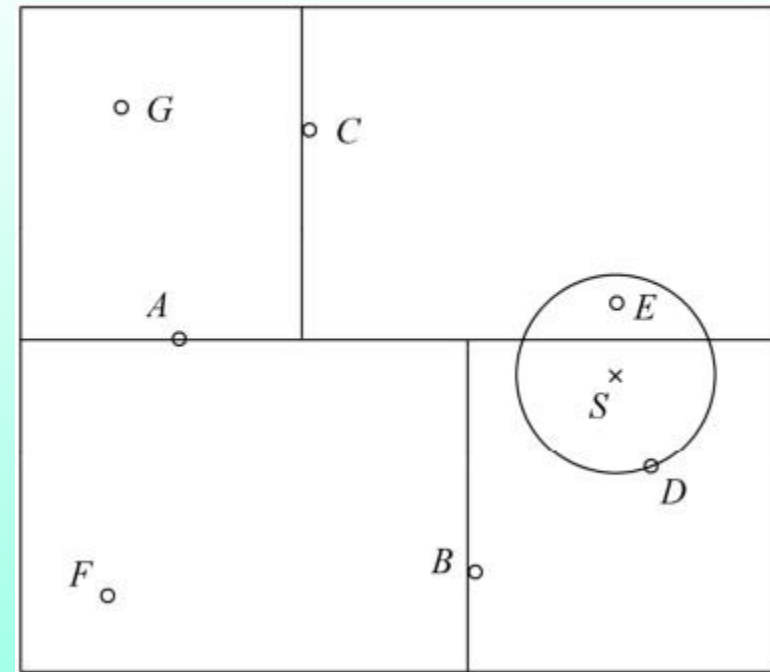
K近邻法的实现:

➤ 搜索kd树

- 算法转到更上一级的父结点，继续上述过程。如果父结点的另一子结点的超矩形区域与超球体不相交，或不存在比当前最近点更近的点，则停止搜索。

例：给定一个如图示的kd树，根结点为A，其子结点为B,C等。树上共存储7个实例点；另有一个输入目标实例点S，求S的最近邻

解：首先在kd树中找到包含S的叶结点D，以点D作为近似最邻。真正最近邻一定在以点S为中心通过点D的圆的内部。然后返回结点D的父结点B，在结点B的另一子结点F的区域内搜索最近邻。结点F的区域与圆不相交，不可能有最近邻。继续返回上一级父结点A，在结点A的另一子结点C的区域内搜索最近邻。结点C的区域与圆相交；该区域在圆内的实例点有点E，点E比点D更近，成为新的最近邻近似。最后得到点E是点S的最近邻。



Summary

- IBL :
 - 属消极学习方法
 - 无需构造目标函数的显式假设
 - 为每个查询点构造目标函数的不同逼近
- IBL优点：
 - 用不太复杂的局部逼近函数集构造复杂目标函数
 - 不会损失训练样例的任何信息
- IBL问题：
 - 分类效率低
 - 合适的实例距离度量函数难构造
 - 无关特征对距离的副作用

Summary (2)

- k-Nearest Neighbor
- Locally weighted regression methods
 - k-NN的推广
- Radial basis function networks
 - 局部核函数构成的神经网络
 - 可看作IBL 及ANN的混合

- 作业： 8.1